

1. Welcome

Customize the **Mobile Status & Navigation bar**, it offers a range of features and options to tailor the appearance and behavior of the bars.

Visit the [Unity Asset Store](#) to access this asset.

- [Setup](#): Find detailed instructions on how to set up and configure the asset in your Unity project.
- [C# Code](#): Explore the code [references](#) to facilitate customization and extend functionality.
- [Videos](#): Watch informative videos demonstrating various techniques and showcasing the asset's capabilities.

Take control of the mobile status and navigation bar in your Unity projects and create immersive mobile experiences that stand out.

Mobile Status & Navigation bar



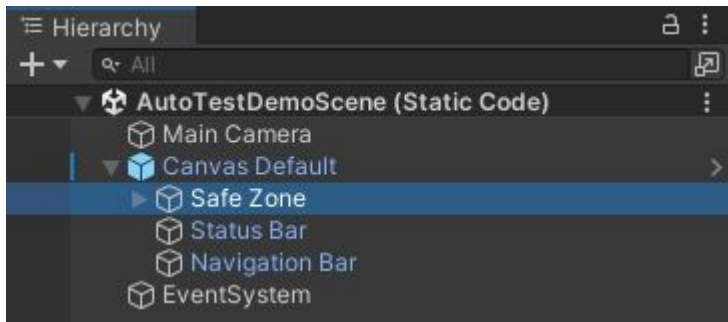
Supported platforms:
Android & iOS

2. Setup

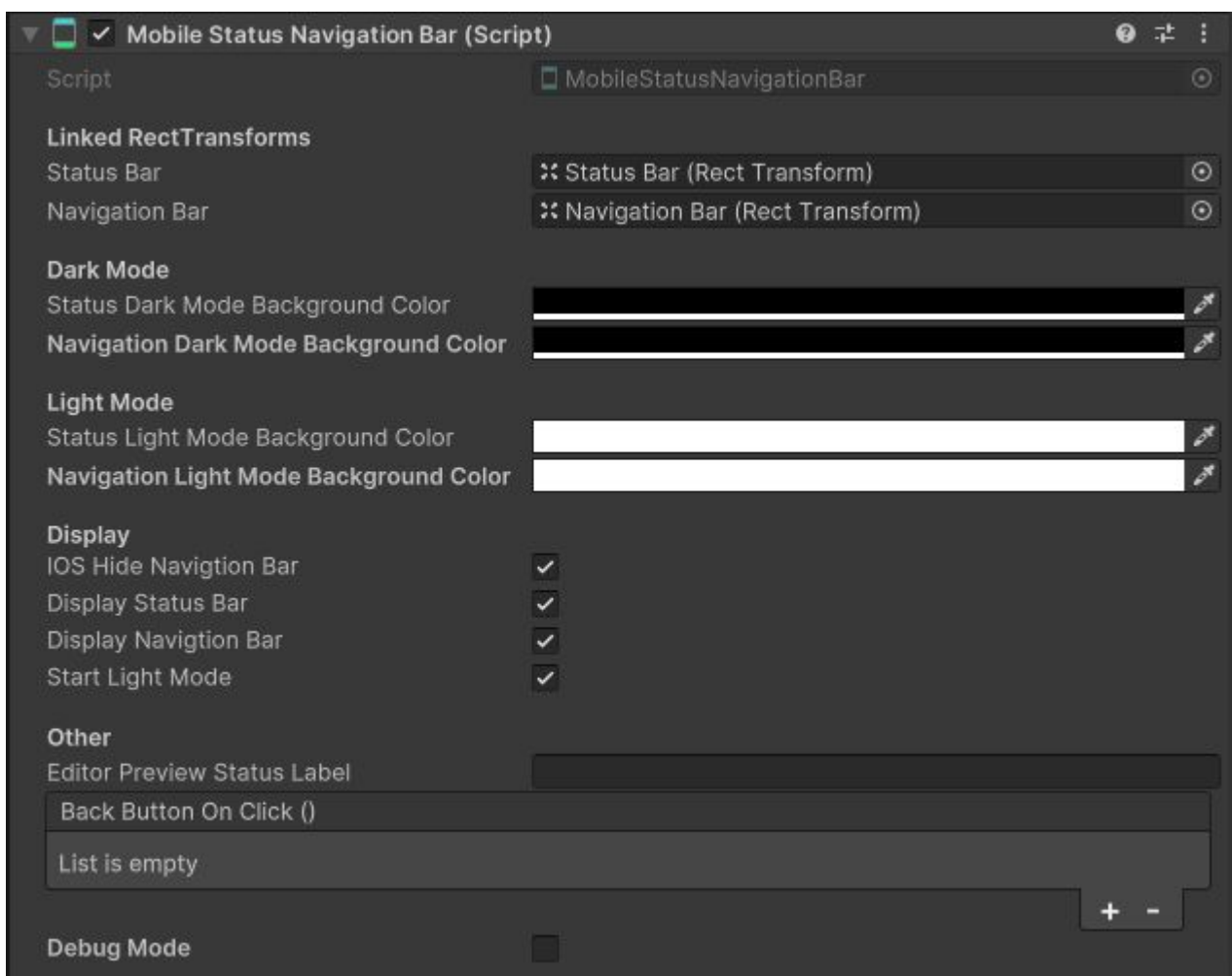
Canvas:

To get started, add either the **Canvas Default** or **Canvas Scroll View** prefab from **Assets/Saari.Tech/StatusNavigation/Prefabs/** to your scene. Don't forget to include the **EventSystem** as well.

You can find the **Mobile Status Navigation Bar** component within the hierarchy of either **Canvas Default/Safe Zone** or **Canvas Scroll View/Safe Zone**



The **Mobile Status Navigation Bar** component looks like this:



The component provides the following features:

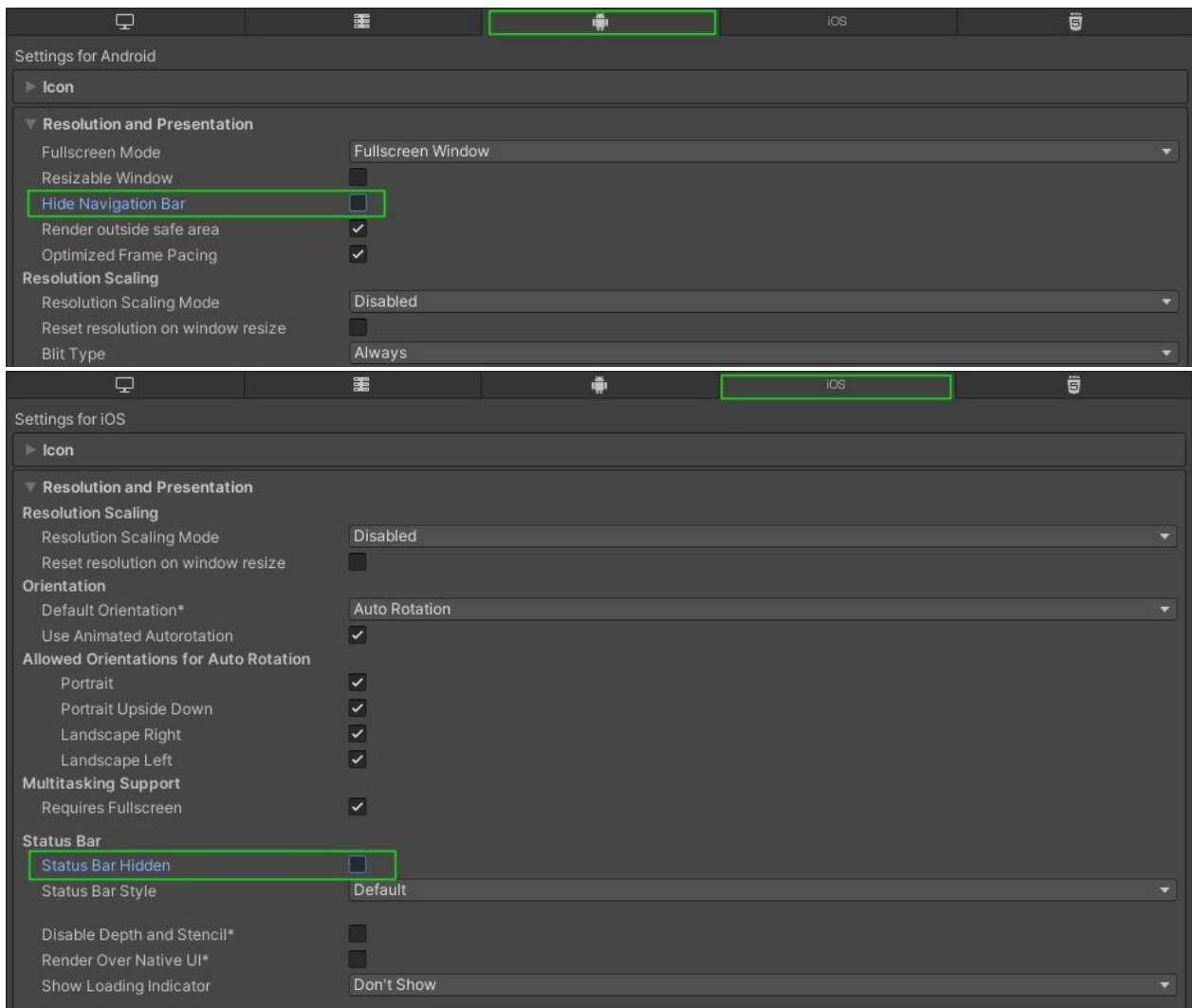
1. **Linked RectTransforms:** Connected status bar and navigation bar.
2. **Dark Mode:** Set the background color for the bars to dark mode.
3. **Light Mode:** Set the background color for the bars to light mode.
4. **Display Status & Navigation:** Enable this option to show both or one of the bars in your application.
5. **Start Light Mode:** Disable this option to start in dark mode.
6. **Editor Preview Status Label:** By default, it displays "Editor".
7. **Back Button On Click:** Define the code to be executed when the mobile back button is clicked.
8. **Debug Mode:** Gives a log of what actually happened, useful for debugging.

Improve the User Experience at Startup (Optional):

To ensure the smooth display of the status and navigation bar from the outset of your app, without any abrupt appearance after the **splash screen**, follow these steps:

1. **Android:** In the Android Player Settings, navigate to the **Resolution and Presentation** section. Make sure that the **Hide Navigation Bar** option is unchecked.
2. **iOS:** For iOS devices, access the iOS Player Settings and proceed to the **Resolution and Presentation** settings. Confirm that the **Status Bar Hidden** option is also unchecked.

By configuring these settings appropriately (*see images in next page*), you'll be able to establish your desired status and navigation bar appearance right from the splash screen's commencement. With these settings, you will be able to set up your Status & Navigation bar during Splash Screen later in the [C# Code Part 2](#).



These features combine to create a versatile and customizable for your application. Next, let's integrate the [code](#) to access additional functions and unlock more capabilities.

3. C# Code Part 1

To utilize the functionality of **Mobile Status & Navigation Bar** in your code, include the following line:

```
using SaariTech.UI;
```

You have two options for accessing the component:

1. Accessing the component statically: You can directly access the component from the current activated GameObject and its methods without adding the object to your code. For example:

```
// Execute the code statically  
StatusNavigation.Show();
```

2. Using the component as a reference: Instantiate an object of the component class and access its methods and properties. For example:

```
// Get the component  
MobileStatusNavigationBar mobileStatusNavigationBar =  
GetComponent<MobileStatusNavigationBar>();  
  
// Execute the code dynamically  
mobileStatusNavigationBar.Hide();
```

Here are the C# public functions you can use in both dynamic and static method:

To show and hide the status and navigation bar:

```
// ≡ Status & Navigation ≡  
StatusNavigation.Show();  
StatusNavigation.Hide();  
  
// ≡ Status ≡  
StatusNavigation.ShowStatus();  
StatusNavigation.HideStatus();  
  
// ≡ Navigation ≡  
StatusNavigation.ShowNavigation();  
StatusNavigation.HideNavigation();
```

To set the background color of the status and navigation bars:

```
// == Status & Navigation ==
StatusNavigation.SetColor(Color color);

// == Status ==
StatusNavigation.SetStatusColor(Color color);

// == Navigation ==
StatusNavigation.SetNavigationColor(Color color);
```

To set the background color to light or dark mode:

```
// == Status & Navigation ==
StatusNavigation.SetLightMode();
StatusNavigation.SetDarkMode();

// == Status ==
StatusNavigation.SetStatusLightMode();
StatusNavigation.SetStatusDarkMode();

// == Navigation ==
StatusNavigation.SetNavigationLightMode();
StatusNavigation.SetNavigationDarkMode();
```

To modify the color of the light and dark mode templates:

```
// == Status & Navigation ==
StatusNavigation.ChangeLightModeColorTemplate(Color color);
StatusNavigation.ChangeDarkModeColorTemplate(Color color);

// == Status ==
StatusNavigation.ChangeStatusLightModeColorTemplate(Color color);
StatusNavigation.ChangeStatusDarkModeColorTemplate(Color color);

// == Navigation ==
StatusNavigation.ChangeNavigationLightModeColorTemplate(Color color);
StatusNavigation.ChangeNavigationDarkModeColorTemplate(Color color);
```

Please note that these methods will update the color template, you still need to run **StatusNavigation.SetLightMode();** or **StatusNavigation.SetDarkMode();** or other **Light & Dark Mode Toggle** to apply the changes and update the color of the system bars accordingly.

If you're seeking more **advanced functionalities**, you may find the [C# Code Part 2](#) useful. It offers additional resources and information that can further extend your options with the Asset.

4. C# Code Part 2

if-statement and callback:

```
void Start()
{
    if (StatusNavigation.Show(ToggleCompleted))
    {
        Debug.Log("The system bars are ready and toggling now.");
    }
    else
    {
        Debug.Log("The system bars are not ready.");
    }
}

void ToggleCompleted()
{
    Debug.Log("Toggled!");
}
```

Apply color transitions to the bars:

```
// == Status & Navigation ==
StatusNavigation.TransitionColor(Color color, float seconds);
StatusNavigation.TransitionLightMode(float seconds);
StatusNavigation.TransitionDarkMode(float seconds);

// == Status ==
StatusNavigation.TransitionStatusColor(Color color, float seconds);
StatusNavigation.TransitionStatusLightMode(float seconds);
StatusNavigation.TransitionStatusDarkMode(float seconds);

// == Navigation ==
StatusNavigation.TransitionNavigationColor(Color color, float seconds);
StatusNavigation.TransitionNavigationLightMode(float seconds);
StatusNavigation.TransitionNavigationDarkMode(float seconds);
```

Get color of status bar:

```
Color statusLightColor = StatusNavigation.statusLightModeBackgroundColor;
Color statusDarkColor = StatusNavigation.statusDarkModeBackgroundColor;
```

Get color of navigation bar:

```
Color navigationLightColor = StatusNavigation.navigationLightModeBackgroundColor;
Color navigationDarkColor = StatusNavigation.navigationDarkModeBackgroundColor;
```

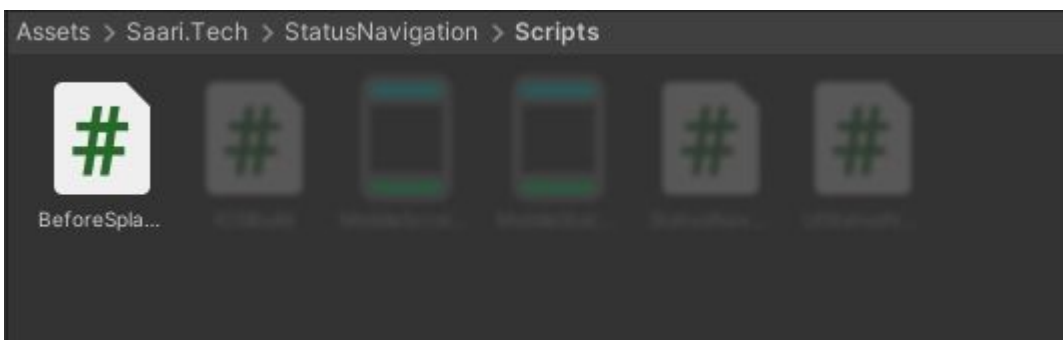
Get height of status and navigation bar:

```
// == Realtime Data ==  
float statusBarHeight = StatusNavigation.StatusBarHeight;  
float navigationBarHeight = StatusNavigation.NavigationBarHeight;  
  
// == Static Data ==  
float staticStatusBarHeight = StatusNavigation.StatusBarHeight_STATIC;  
float staticNavigationBarHeight = StatusNavigation.NavigationBarHeight_STATIC;
```

Other:

```
UnityEvent backButtonEvent = StatusNavigation.backButtonOnClick;  
bool startLightMode = StatusNavigation.startLightMode;  
bool statusBarIsReady = StatusNavigation.IsReadyStatusBar;  
bool navigationIsReady = StatusNavigation.IsReadyNavigationBar;
```

Set system bars during the Splash Screen by editing this C# file:



Before using this code, ensure that **Hide Navigation Bar** option is **unchecked** in the Resolution and Presentation settings of Android Player Settings and that **Status Bar Hidden** option is **unchecked** in the Resolution and Presentation settings of iOS Player Settings.

The provided code, located in the **BeforeSplashScreen.cs** file, it overrides specific aspects of the system bars during the Splash Screen phase. However, be aware that this code has limited functionality and may not cover all scenarios or requirements.

```
// == Display ==  
internal static bool BeforeSplashScreenDisplayStatusBar = true;  
internal static bool BeforeSplashScreenDisplayNavigationBar = true;  
  
// == Light Mode==  
internal static bool BeforeSplashScreenStatusBarLightMode = true;  
internal static bool BeforeSplashScreenNavigationBarLightMode = true;
```

Feel free to use these code snippets to access and manipulate the status and navigation bar properties in your application.